UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"
CAMPUS DE SOROCABA

CURSO DE ENGENHARIA DE CONTROLE E AUTOMAÇÃO

VITOR CORREIA MENDES

Trabalho Individual II - SCOM

Backend, Persistência e Segurança: ***Cuphead Boss Guide***

SOROCABA
2026

Índice

1 Introdução **2**

2 Evolução em Relação ao Trabalho 1 **2**

3 Requisitos Funcionais **3**

4 Arquitetura do Sistema **3**

5 Modelo de Dados **4**

6 Estratégia de Autenticação **5**

7 Estratégia de Segurança **6**

8 Endpoints Implementados **6**

9 Estratégia de Testes **7**

10 Resultados dos Testes de Segurança **8**

11 Logs e Observabilidade **10**

12 Desempenho e Limitações **10**

13 Uso de IA Generativa **10**

14 Conclusões **11**

15 Anexos (Evidências) **11**

1 Introdução

O presente relatório técnico documenta a evolução do *Cuphead Boss Guide*, desenvolvido no Trabalho Individual I como uma interface estática em HTML5 e CSS3. Nesta segunda etapa, o sistema recebeu um backend funcional com API REST, persistência em banco de dados relacional, autenticação de usuários, controle de acesso por perfis, validação de entradas, proteções de segurança, testes automatizados e registro de logs. O backend foi escrito em JavaScript (Node.js com Express), seguindo a sugestão de *JavaScript everywhere*, e o banco de dados escolhido foi o PostgreSQL.

O código-fonte, o script SQL e este relatório estão no repositório `https://github.com/akavirto123/scom-projeto-cuphead` (tag `v2.0`). As instruções de execução local estão no `README.md`.

2 Evolução em Relação ao Trabalho 1

A seção de limitações do Trabalho I apontava que o formulário "Sugerir Nova Tática" era apenas demonstrativo e que o checklist de conquistas não guardava o progresso do usuário. As duas limitações foram resolvidas: as táticas agora são gravadas no banco e passam por moderação, e cada usuário tem o seu progresso de conquistas salvo.

Item	**Trabalho I**	**Trabalho II**
Chefes	19 cartões fixos no HTML	Vindos do banco, com paginação, filtro por ilha, busca e CRUD do administrador
Formulário de tática	Sem efeito	Grava a tática; o administrador aprova ou rejeita
Conquistas	Lista estática	Progresso salvo por usuário, com percentual
Usuários	Inexistentes	Cadastro, login, logout e três perfis
Páginas	`index.html`	`index.html`, `conta.html`, `admin.html`
Qualidade	Lighthouse e W3C	56 testes automatizados, testes de segurança, logs e benchmark

Tabela 1: Comparação entre as duas versões

A identidade visual também evoluiu. As fontes passaram a ser embutidas no próprio projeto, sem depender do Google Fonts, e foram escolhidas para reforçar a estética dos anos 1930: *Fascinate* no logotipo, *Rye* nos títulos e botões (estilo cartaz de circo e cassino), *Libre Baskerville* no texto corrido e *Special Elite* (máquina de escrever) em rótulos e legendas. Foram acrescentados um letreiro de lâmpadas no cabeçalho, textura de papel envelhecido, moldura de película no destaque e na galeria e cartões em formato de ingresso. Todas as animações são desligadas quando o sistema operacional pede movimento reduzido.



Figura 1: Página inicial do Trabalho II: chefes carregados da API e conquistas

3 Requisitos Funcionais

ID	Requisito	Perfil
RF01	Listar chefes com paginação, filtro por ilha e busca	Todos
RF02	Ver as táticas aprovadas de cada chefe	Todos
RF03	Criar conta, entrar e sair	Visitante / logado
RF04	Enviar tática para um chefe (fica pendente)	Jogador, admin
RF05	Editar ou excluir as próprias táticas enquanto pendentes	Jogador, admin
RF06	Marcar conquistas e ver o percentual concluído	Jogador, admin
RF07	Cadastrar, editar e excluir chefes	Admin
RF08	Moderar táticas (aprovar, rejeitar, excluir)	Admin
RF09	Listar usuários e alterar o perfil de outro usuário	Admin

Tabela 2: Requisitos funcionais

4 Arquitetura do Sistema

Adotou-se uma arquitetura cliente-servidor com o backend organizado em **camadas**. O frontend (pasta `frontend/`) é composto pelas páginas HTML do Trabalho I e por módulos JavaScript que consomem a API. O backend (pasta `backend/src/`) separa a interface HTTP (`routes/` e `controllers/`), a lógica de aplicação (`services/`, `domain/` e

`validators/`) e o acesso a dados (`repositories/`). Cada camada só conhece a camada de baixo: controllers não escrevem SQL e repositories não conhecem requisições HTTP.

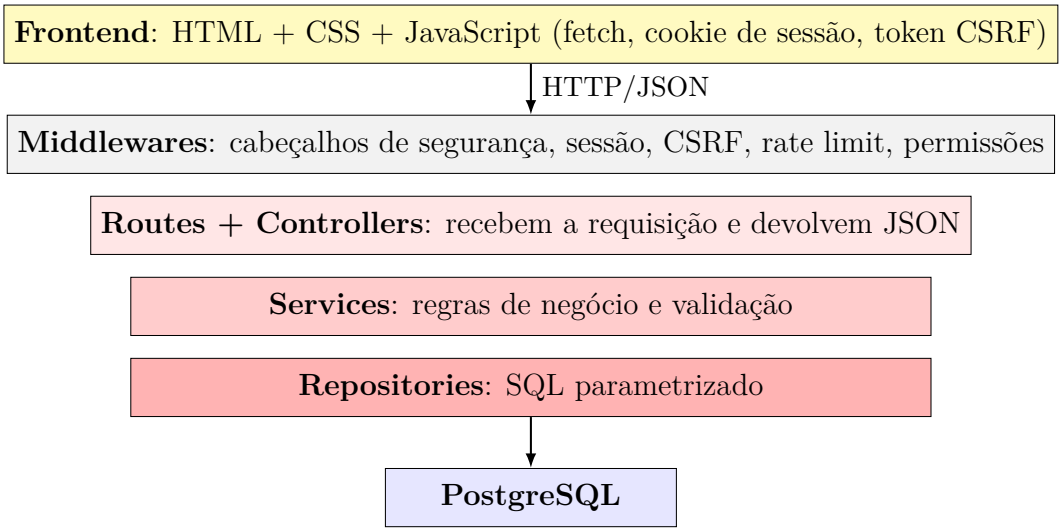


Figura 2: Camadas da aplicação

O arquivo `app.js` é o único ponto que cria e conecta as camadas: cada service recebe os repositories de que precisa (injeção de dependências), o que permite testar a lógica de negócio com repositórios falsos. A configuração fica separada do código em `src/config/env.js`, lida de um arquivo `.env` que não vai para o Git (o repositório contém apenas o modelo `.env.example`). Erros de qualquer camada são tratados em um único middleware (`tratadorErros.js`). A descrição completa da arquitetura está em `docs/ARQUITETURA.md`.

5 Modelo de Dados

O banco possui oito tabelas. A entidade principal é `chefes`; `taticas` guarda as sugestões da comunidade e `usuario_conquistas` o progresso de cada usuário. As tabelas `sessoes` e `schema_migrations` servem, respectivamente, às sessões de login e ao controle de migrações.

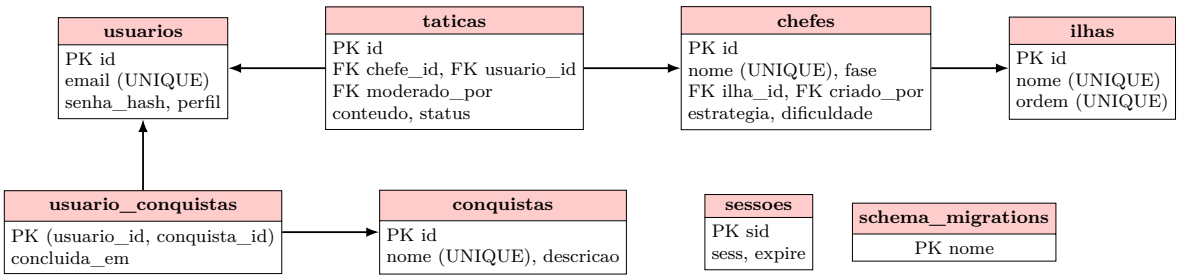


Figura 3: Modelo de dados (as setas apontam para a tabela referenciada)

Todas as tabelas têm chave primária. As sete chaves estrangeiras usam `ON DELETE CASCADE` para dados dependentes (táticas e progresso), `RESTRICT` para a ilha de um

chefe e `SET NULL` para autor e moderador. Há restrições `UNIQUE` (e-mail, nome do chefe, nome da conquista), `NOT NULL` em todos os campos obrigatórios e `CHECK` para perfil, status, dificuldade de 1 a 5 e tamanho dos textos. Foram criados seis índices para as consultas mais usadas (chefes por ilha, busca por nome, táticas por chefe e status, táticas por usuário, fila de moderação e expiração de sessões).

O esquema é versionado por **migrações** (`database/migrations/`): `001` cria as tabelas e `002` importa os 19 chefes e 28 conquistas que antes estavam fixos no HTML. O comando `npm run db:migrar` aplica apenas o que falta, e o arquivo `database/script_completo.sql`, gerado a partir das migrações, é o script de criação do banco, que roda sem erros direto no `psql` ou no pgAdmin.

6 Estratégia de Autenticação

Optou-se por **sessão no servidor** em vez de JWT, porque assim o logout invalida o acesso imediatamente. O navegador guarda apenas um identificador em um cookie `HttpOnly` (inacessível ao JavaScript) e `SameSite=Strict`; os dados da sessão ficam na tabela `sessoes`. As senhas são armazenadas apenas como hash **bcrypt** com custo 12. No login, o id de sessão é regenerado (contra fixação de sessão) e a mensagem de erro é a mesma para e-mail inexistente ou senha errada. No logout, a sessão é apagada do banco e o cookie antigo deixa de funcionar. Tentativas de login malsucedidas são limitadas a 5 por IP a cada 15 minutos.

O sistema tem três perfis: **visitante** (sem login), **jogador** (padrão no cadastro) e **administrador**. A matriz de permissões está em um único arquivo (`src/domain/permissoes.js`) e cada rota declara a ação que exige. Sem login, a resposta é 401; com perfil insuficiente, 403.

Ação	**Visitante**	**Jogador**	**Admin**
Ver chefes, ilhas, conquistas e táticas aprovadas	✓	✓	✓
Enviar tática	–	✓	✓
Editar/excluir a própria tática (só pendente)	–	✓	✓
Marcar o próprio progresso de conquistas	–	✓	✓
Criar, editar e excluir chefes	–	–	✓
Moderar táticas (aprovar, rejeitar, excluir qualquer uma)	–	–	✓
Listar usuários e alterar perfil de outro usuário	–	–	✓

Tabela 3: Matriz de permissões

7 Estratégia de Segurança

Ameaça	**Proteção adotada**
SQL Injection	Todas as consultas usam parâmetros (`$1, $2, ...`); a entrada nunca é concatenada ao SQL.
XSS	A API responde só JSON com `nosniff`; o frontend insere dados apenas como texto (`textContent`, nunca `innerHTML`); *Content Security Policy* bloqueia scripts inline.
CSRF	Token aleatório por sessão exigido no cabeçalho `X-CSRF-Token` em POST, PUT, PATCH e DELETE, além do cookie `SameSite=Strict`.
Acesso indevido	Middleware de permissão em todas as rotas protegidas e verificação de autoria nas táticas.
Força bruta	*Rate limiting* no login (5 falhas/15 min) e na API (300 req/min).
Vazamento de dados	Stack trace só em modo de desenvolvimento; respostas nunca trazem `senha_hash`; segredos fora do código (`.env`).

Tabela 4: Ameaças e proteções

Validação e sanitização. Toda entrada externa passa pela função `validar()`, que confere o tipo de cada campo (texto, inteiro, e-mail, senha, opção), o tamanho mínimo e máximo e os campos obrigatórios. Campos não previstos são descartados, então enviar `"perfil": "admin"` no cadastro não tem efeito. Os textos são sanitizados: espaços repetidos e caracteres de controle ou invisíveis são removidos. Os erros voltam estruturados, um por campo, como em `{"campo": "email", "mensagem": "deve ser um e-mail válido."}`.

Tratamento de erros. Toda falha termina no middleware central, que responde com status HTTP, código, mensagem legível e um `requisicaoId`. O bloco com detalhes técnicos (stack trace) só aparece quando `NODE_ENV=development`.

8 Endpoints Implementados

A API possui 25 endpoints. A documentação completa, com parâmetros, exemplos de requisição e de resposta e códigos de status de cada um, está em `docs/API.md` (Anexo B). Todas as listagens aceitam paginação (`?pagina=` e `?limite=`).

Método	URL	Acesso	Status
GET	`/api/saude`	público	200, 500
GET	`/api/auth/sessao`	público	200
POST	`/api/auth/registro`	público	201, 403, 409, 422
POST	`/api/auth/login`	público	200, 401, 403, 422, 429
POST	`/api/auth/logout`	logado	204, 401, 403
GET	`/api/auth/me`	logado	200, 401
GET	`/api/ilhas`	público	200
GET	`/api/chefes`	público	200, 422
GET	`/api/chefes/:id`	público	200, 404, 422
POST	`/api/chefes`	admin	201, 401, 403, 409, 422
PUT	`/api/chefes/:id`	admin	200, 401, 403, 404, 409, 422
DELETE	`/api/chefes/:id`	admin	204, 401, 403, 404
GET	`/api/chefes/:id/taticas`	público	200, 404
POST	`/api/taticas`	jogador, admin	201, 401, 403, 422
GET	`/api/taticas/minhas`	jogador, admin	200, 401
PATCH	`/api/taticas/:id`	autor	200, 401, 403, 404, 409, 422
DELETE	`/api/taticas/:id`	autor ou admin	204, 401, 403, 404, 409
GET	`/api/taticas`	admin	200, 401, 403
PATCH	`/api/taticas/:id/status`	admin	200, 401, 403, 404, 422
GET	`/api/conquistas`	público	200
GET	`/api/conquistas/progresso`	jogador, admin	200, 401
PUT	`/api/conquistas/:id/progresso`	jogador, admin	204, 401, 404
DELETE	`/api/conquistas/:id/progresso`	jogador, admin	204, 401, 404
GET	`/api/usuarios`	admin	200, 401, 403
PATCH	`/api/usuarios/:id/perfil`	admin	200, 401, 403, 404, 409

Tabela 5: Resumo dos endpoints

9 Estratégia de Testes

Os testes usam o executor nativo do Node.js (`node:test`) e a biblioteca supertest, que envia requisições HTTP reais para a aplicação. Foram escritos **56 testes automatizados** em dois níveis:

- **Unitários (34):** testam a lógica de negócio sem banco de dados, usando repositórios falsos em memória: validação, matriz de permissões e os services de autenticação, chefes e táticas.
- **Integração (22):** sobem a aplicação completa contra um banco de testes separado, recriado a cada execução, e cobrem o CRUD, o fluxo de moderação, as conquistas, o formato dos erros e os testes de segurança.

Os casos inválidos incluem dados fora do limite, tipos errados, e-mail e nome duplicados, senha incorreta, tática de outro usuário, tática já moderada, JSON malformado e rota inexistente. Todos os testes rodam com `npm test` em cerca de 3 segundos; o resultado está no Anexo C.

10 Resultados dos Testes de Segurança

Os ataques abaixo foram executados com `curl` contra o servidor local pelo script `backend/scripts/demonstrar-seguranca.sh`. Os mesmos cenários também fazem parte dos testes automatizados.

Teste	Tentativa	Resultado
SQL Injection	`' OR '1'='1' --` no login e `'; DROP TABLE chefes; --` na busca	Login recusado (422/401); busca tratada como texto e tabela intacta
XSS	Tag `<script>` com `alert(document.cookie)` em uma tática	Exibido como texto, sem execução
CSRF	POST com cookie válido e sem token	403 `CSRF_INVALIDO`
Acesso sem login	`GET /api/taticas/minhas`	401 `NAO_AUTENTICADO`
Perfil insuficiente	Jogador excluindo chefe e listando usuários	403 `ACESSO_NEGADO`
Sessão inválida	Reusar o cookie depois do logout	401
Força bruta	Várias senhas erradas seguidas	429 após 5 falhas

Tabela 6: Resumo dos testes de segurança

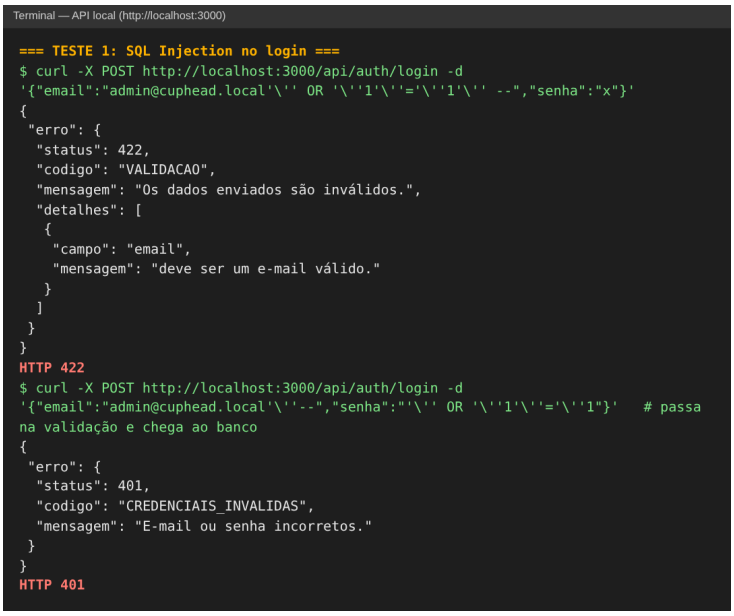
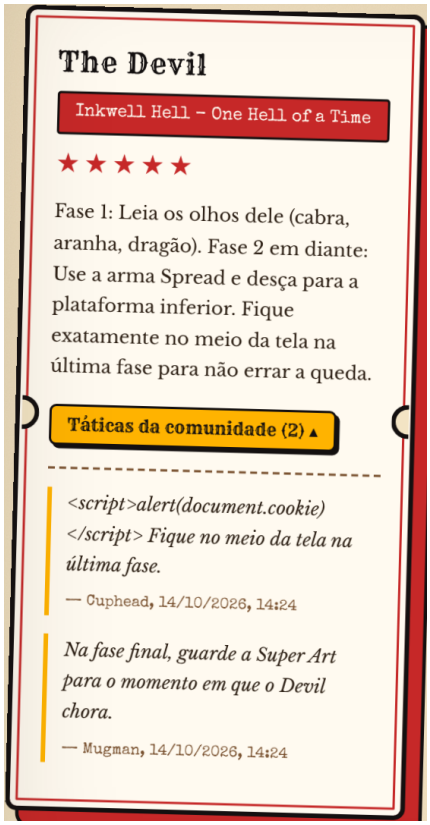


Figura 4: SQL Injection no login: barrada pela validação e, quando chega ao banco, tratada como texto





Figura 5: SQL Injection na busca (esquerda) e requisição sem token CSRF (direita)



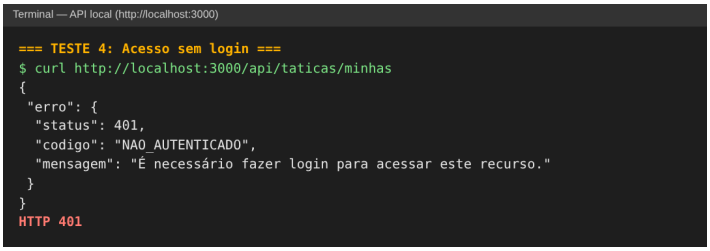
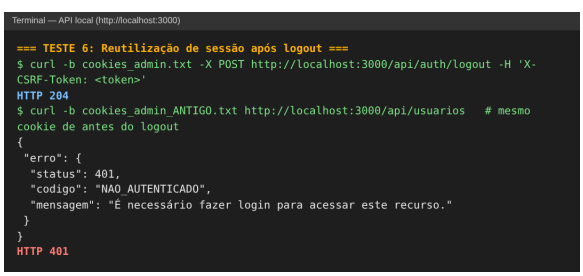




Figura 6: XSS exibido como texto no navegador (esquerda); acesso sem login e com perfil insuficiente (direita)



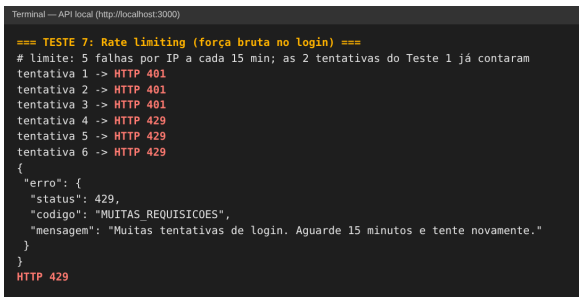


Figura 7: Cookie reutilizado após o logout (esquerda) e *rate limiting* do login (direita)

11 Logs e Observabilidade

O módulo `config/logger.js` grava cada evento como uma linha JSON no arquivo `backend/logs/app.log`, sempre com *timestamp*. São registrados, entre outros, login bem-sucedido (`auth.login_sucesso`), falha de autenticação (`auth.login_falha`), criação de entidade (`entidade.criada`), erro interno (`erro.interno`), acesso negado e tentativa de CSRF. Senhas, tokens e cookies nunca são gravados. Cada requisição recebe um identificador que aparece no log e na resposta de erro, o que liga uma falha relatada pelo usuário à sua causa. Trecho real do log:


```
{"timestamp":"2026-10-14T17:24:15.001Z","nivel":"info","evento":"auth.login_sucesso","usuarioId":2,"perfil":"
     jogador","ip":"127.0.0.1"}
{"timestamp":"2026-10-14T17:24:17.781Z","nivel":"aviso","evento":"auth.login_falha","email":"admin@cuphead.
     local","ip":"127.0.0.1","motivo":"senha_incorreta"}
{"timestamp":"2026-10-14T17:24:15.045Z","nivel":"info","evento":"entidade.criada","entidade":"tatica","id":1,"
     chefeId":1,"por":2}
{"timestamp":"2026-10-14T17:27:56.882Z","nivel":"erro","evento":"erro.interno","requisicaoId":"b9c70b6a-5ccb
     -4876-bfc8-3ecd5bfe2027","metodo":"GET","rota":"/api/saude","mensagem":"connect ECONNREFUSED
     127.0.0.1:5432"}
```


12 Desempenho e Limitações

O desempenho foi medido com o comando `npm run bench`, que usa a ferramenta autocannon com 10 conexões simultâneas durante 5 segundos por endpoint.

Endpoint	Req/s	Latência média	p99
`GET /api/chefes`	537	18,1 ms	45 ms
`GET /api/chefes/1`	996	9,5 ms	22 ms
`GET /api/chefes/1/taticas`	975	9,8 ms	19 ms
`GET /api/conquistas`	1219	7,7 ms	16 ms

Tabela 7: Resultados do benchmark (nenhum erro)

Todas as leituras ficaram abaixo de 50 ms no pior caso. O login é mais lento de propósito: o bcrypt leva cerca de 0,5 s por verificação, o que torna inviável testar milhões de senhas caso o banco vaze.

Como limitações, o *rate limiting* é guardado em memória (zera quando o servidor reinicia); o sistema roda em HTTP localmente e precisaria de HTTPS em produção; não há recuperação de senha por e-mail; e as imagens da galeria continuam hospedadas em sites externos, como no Trabalho I.

13 Uso de IA Generativa

Ferramenta Utilizada: Claude (Anthropic). No Trabalho I havia sido usado o Google Gemini.

Finalidade: a IA recebeu o enunciado, o relatório e o código do Trabalho I e foi usada para planejar e gerar a maior parte do Trabalho II: o backend, o modelo de dados e as migrações, o JavaScript e o novo visual do frontend, os testes automatizados, a

documentação (README e API) e o texto deste relatório. Todos os commits a partir de `chore: reorganiza o projeto do T1 na pasta frontend/` foram feitos com apoio da ferramenta (Anexo F).

Problemas Encontrados e Corrigidos: ao executar e testar o sistema, apareceram falhas no que foi gerado, por exemplo: a senha do administrador com `#` era cortada pelo arquivo `.env`; o comando que recria o banco não funcionava em versões antigas do PostgreSQL; os chefes eram inseridos fora da ordem do guia; mensagens tinham erro de concordância ("Conquista não encontrado"); e, no celular, o cabeçalho fixo passou a ocupar metade da tela.

Avaliação Crítica: a IA acelerou bastante a escrita de código repetitivo e sugeriu proteções que não estavam no enunciado, como a comparação de senha em tempo constante. Por outro lado, os problemas acima só apareceram quando o sistema foi executado de verdade, o que mostra que o código gerado precisa ser testado e revisado antes de ser aceito.

14 Conclusões

O *Cuphead Boss Guide* deixou de ser uma página estática e passou a ser uma aplicação cliente-servidor completa, com dados persistentes, usuários com perfis diferentes e um fluxo de colaboração (envio e moderação de táticas). As duas limitações apontadas no Trabalho I foram resolvidas, e a interface ganhou uma identidade visual mais fiel à estética dos anos 1930.

Além dos requisitos obrigatórios, foram implementadas as cinco boas práticas opcionais: configuração por `.env`, migrações de banco, paginação, *rate limiting* e testes de integração automatizados. A separação em camadas foi o que permitiu testar a lógica de negócio isoladamente e, ao mesmo tempo, testar o sistema completo contra um banco real. Como trabalhos futuros, ficam a publicação com HTTPS, a recuperação de senha e uma nova auditoria Lighthouse da interface.

15 Anexos (Evidências)

A. Script SQL

O script completo de criação do banco está em `backend/database/script_completo.sql`. Trecho da tabela principal:

```
CREATE TABLE chefes (
    id             INTEGER GENERATED ALWAYS AS IDENTITY,
    nome           VARCHAR(60)  NOT NULL,
    fase           VARCHAR(80)  NOT NULL,
    ilha_id        SMALLINT     NOT NULL,
    estrategia     VARCHAR(1000) NOT NULL,
    dificuldade    SMALLINT     NOT NULL,
    criado_por     INTEGER,
    criado_em      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    atualizado_em  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT pk_chefes PRIMARY KEY (id),
    CONSTRAINT uq_chefes_nome UNIQUE (nome),
    CONSTRAINT fk_chefes_ilha FOREIGN KEY (ilha_id)
        REFERENCES ilhas (id) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_chefes_criado_por FOREIGN KEY (criado_por)
```

```
        REFERENCES usuarios (id) ON DELETE SET NULL,
    CONSTRAINT ck_chefes_dificuldade CHECK (dificuldade BETWEEN 1 AND 5),
    CONSTRAINT ck_chefes_estrategia_tamanho CHECK (char_length(estrategia) BETWEEN 20 AND 1000)
);
CREATE INDEX idx_chefes_ilha_id ON chefes (ilha_id);
```

B. Documentação da API

A documentação completa está em `docs/API.md`. Exemplo de um endpoint:

```
POST /api/taticas            (jogador ou admin; cabeçalho X-CSRF-Token obrigatório)
Corpo: { "chefeId": 1, "conteudo": "Fique no canto durante a fase da cenoura." }
  chefeId  inteiro, chefe existente      conteudo  texto, 10 a 1000 caracteres

201 Created  ->  { "dados": { "id": 1, "conteudo": "Fique no canto ...", "status": "pendente",
                              "chefeId": 1, "chefeNome": "The Root Pack", "autorNome": "Cuphead" } }
422          ->  { "erro": { "status": 422, "codigo": "VALIDACAO",
                             "detalhes": [ { "campo": "chefeId", "mensagem": "chefe inexistente." } ] } }
Outros: 401 (sem login), 403 (sem token CSRF)
```

C. Resultado dos Testes Automatizados



Figura 8: Execução de `npm test`: 56 testes aprovados, nenhuma falha

D. Capturas dos Testes de Segurança

Apresentadas na seção 10. Os textos completos estão em `docs/evidencias/seguranca_manual.txt`.

E. Amostra dos Logs

Apresentada na seção 11. Amostra maior em `docs/evidencias/logs_amostra.txt`.

F. Histórico Git

O desenvolvimento usou branches por etapa (`feature/*`, `test/*`, `docs/*`) integradas em `develop` e depois em `main`, com tags `v1.0` (Trabalho I) e `v2.0` (Trabalho II) e mensagens no padrão *Conventional Commits*.

```
*   7543a21 (HEAD -> main) release: Trabalho Individual II com visual vintage e relatório final
|\
| *   ec7ed1c (develop) merge: docs/relatorio-enxuto em develop
| |\
| | * 145deed (docs/relatorio-enxuto) docs(relatorio): reescreve relatório no formato do T1 e atualiza capturas
| |/
| * 3dd1798 merge: feature/visual-vintage em develop
|/|
| * 2750cf5 (feature/visual-vintage) feat(frontend): novo visual vintage com fontes embutidas
|/
* df0de6a docs(relatorio): atualiza histórico Git no anexo F
*   62749e0 release: Trabalho Individual II (backend, banco, autenticação e testes)
|\
| *   de675a1 merge: docs/relatorio em develop
| |\
| | * 1c95110 (docs/relatorio) docs(relatorio): adiciona relatório técnico em LaTeX e PDF
| |/
| *   e5db18a merge: docs/documentacao em develop
| |\
| | * fdd2e1e (docs/documentacao) docs: adiciona evidências de testes, logs, desempenho e checklist
| | * a53e38c fix(frontend): ajusta cabeçalho no celular e colunas de ações das tabelas
| | * 57f3e02 docs: documenta API, arquitetura e instruções de execução
| | * 2358233 fix(db): reset das migrações remove apenas objetos do projeto
| |/
| *   df599de merge: test/testes-automatizados em develop
| |\
| | * 05580e3 (test/testes-automatizados) test: adiciona testes unitários, de integração e de segurança
| |/
| *   677cd9d merge: feature/frontend-integracao em develop
| |\
| | * 06887a9 (feature/frontend-integracao) feat(frontend): integra a interface do T1 com a API
| |/
| *   5f7fd02 merge: feature/taticas-conquistas em develop
| |\
| | * ec394c8 (feature/taticas-conquistas) feat(taticas): persiste táticas da comunidade e progresso de conquistas
| |/
| *   0b6f037 merge: feature/api-chefes em develop
| |\
| | * e11f03d (feature/api-chefes) feat(chefes): adiciona CRUD de chefes com paginação, filtros e perfis
| |/
| *   3b46287 merge: feature/autenticacao em develop
| |\
| | * 9f99075 (feature/autenticacao) feat(auth): implementa cadastro, login e logout com sessão segura
| |/
| *   4b284ca merge: feature/banco-de-dados em develop
| |\
| | * 6c47211 (feature/banco-de-dados) feat(db): cria modelo de dados com migrações e script SQL completo
| |/
| *   3f4901c merge: feature/estrutura-backend em develop
| |\
| | * b76af74 (feature/estrutura-backend) feat(backend): cria estrutura base da API com Express
| |/
| * 0bcde00 chore: reorganiza o projeto do T1 na pasta frontend/
|/
* 8887065 (tag: v1.0, origin/main, origin/HEAD) Formatação do README.md
| *-.   af53215 (refs/stash) WIP on docs/relatorio: 0853e19 merge: docs/documentacao em develop
| |\ \
| | | * 00ae889 untracked files on docs/relatorio: 0853e19 merge: docs/documentacao em develop
| | * 9afd585 index on docs/relatorio: 0853e19 merge: docs/documentacao em develop
| |/
| *   16ebb42 merge: docs/documentacao em develop
| |\
| | * 72b0636 docs: adiciona evidências de testes, logs, desempenho e checklist
| | * b756feb fix(frontend): ajusta cabeçalho no celular e colunas de ações das tabelas
| | * b711338 docs: documenta API, arquitetura e instruções de execução
| | * 895be3a fix(db): reset das migrações remove apenas objetos do projeto
| |/
| *   4e14743 merge: test/testes-automatizados em develop
| |\
| | * 0f257a5 test: adiciona testes unitários, de integração e de segurança
| |/
| *   2d34bc6 merge: feature/frontend-integracao em develop
| |\
| | * 0e2087e feat(frontend): integra a interface do T1 com a API
| |/
| *   eaa1d07 merge: feature/taticas-conquistas em develop
| |\
| | * 9ef77d8 feat(taticas): persiste táticas da comunidade e progresso de conquistas
| |/
| *   bdebbef merge: feature/api-chefes em develop
| |\
| | * 0b874f3 feat(chefes): adiciona CRUD de chefes com paginação, filtros e perfis
| |/
| *   5309573 merge: feature/autenticacao em develop
| |\
| | * ad93d38 feat(auth): implementa cadastro, login e logout com sessão segura
| |/
| *   dd4c76e merge: feature/banco-de-dados em develop
```

```
| |\
| | * 9695b92 feat(db): cria modelo de dados com migrações e script SQL completo
| |/
| *   69c5515 merge: feature/estrutura-backend em develop
| |\
| | * 8d690b8 feat(backend): cria estrutura base da API com Express
| |/
| * a1dabf3 chore: reorganiza o projeto do T1 na pasta frontend/
| * 4341785 Formatação do README.md
|/
* 0533055 Adição de interatividade tátil nos botões e efeito de rotação nos cards
* 993580c Implementação de interatividade CSS e filtros visuais na galeria
* d3cf033 Remoção de atributo obsoleta (allowtransparency) no iframe para validação W3C
* e97dbd7 Melhoria de performance com lazy loading no iframe e adição de meta description para SEO
* f6c5628 Correção de contraste no título da seção de formulário
* 3fbfd70 Estrutura inicial do HTML e CSS do guia Cuphead
```

G. Checklist de Requisitos

Requisito	**Status**
Separação entre interface, lógica de aplicação e acesso a dados	[X] Implementado
Arquitetura documentada	[X] Implementado
Entidade principal com criar, ler, atualizar e excluir	[X] Implementado
Consultas parametrizadas	[X] Implementado
Constraints PK, FK, UNIQUE e NOT NULL	[X] Implementado
Script de criação do banco	[X] Implementado
Autenticação por sessão segura	[X] Implementado
Senha com hash bcrypt	[X] Implementado
Logout funcional	[X] Implementado
Proteção de rotas autenticadas	[X] Implementado
Dois ou mais perfis e matriz de permissões	[X] Implementado
Validação de tipo e tamanho, sanitização e erros estruturados	[X] Implementado
Proteção contra SQL Injection, XSS e CSRF	[X] Implementado
Bloqueio de acesso não autorizado	[X] Implementado
Erros com status, mensagem e detalhes só em desenvolvimento	[X] Implementado
Documentação de todos os endpoints	[X] Implementado
Pelo menos 5 testes automatizados, com casos inválidos (56)	[X] Implementado
Logs com timestamp (login, falha, criação, erro interno)	[X] Implementado
Git com branches, tags e commits padronizados	[X] Implementado
Sem credenciais no repositório	[X] Implementado
Boas práticas opcionais	
`.env` para configuração	[X] Implementado
Migrações de banco	[X] Implementado
Paginação de resultados	[X] Implementado
Rate limiting simples	[X] Implementado
Testes de integração automatizados	[X] Implementado